

PART 10

SADP

The Structured AI Development Protocol

A codified shape of one project's AI-assisted-development
learnings. Honestly framed.

Acervator Manual v4.1.0 · Part 10 of 14 · standalone document · live software v3.22.23

Also published as the README of github.com/ekthelius/Structured-AI-Development-Protocol-SADP-

Contents

This Part is designed to be read on its own. It is the canonical description of SADP — what it is, where it came from, what it actually contains, and where the framework claim is and is not yet justified. The full Part shipped in two halves: sections 1-6 in v3.22.7, sections 7-14 in v3.22.8. Both halves are now consolidated in this single document.

1. Standalone introduction — what SADP is, who it is for, and what it is not
2. Framework genesis — Session 22 through SADP v1.92
3. Prior art and the honest framework-claim acknowledgment
4. Design principles and the seven invariants
5. Rule taxonomy — tier, category, lock-state, and lineage
6. Every rule R1 through R77 — full descriptions, dependencies, canonical failure-MEMs
7. Episodic Memory + EDIT_LOG + retag-by-rule
8. Archetype Library + persistent leads + documentation-quality pair
9. Labels-vs-derived-truth structural test family
10. Cascade-gate enforcement — PreToolUse hook layer + release gate sovereignty
11. Token Timer per-action accounting + grounding layers
12. Multi-Agent Coordination Spec v0.1
13. Developmental story — sessions, ships, and rule births
14. Glossary (standalone-complete)

Reading order suggestion. If you are new: Section 1 → 3 → 4 → 6. If you came from the Acervator manual proper: Section 2 → 6 → the second-half sections in v3.22.8. If you are evaluating SADP for adoption: Section 3 is the most important — it is the part that tells you what SADP does NOT yet demonstrate.

1. Standalone introduction

What SADP is

SADP — the Structured AI Development Protocol — is the codified shape of one project's learnings about how to develop software with an AI pair-programmer over an extended timeline. As of this writing, it consists of seventy-seven rules (R1 through R77), an append-only event log (`EDIT_LOG.jsonl`), an episodic memory store (`EPISODIC_MEMORY.json`, 458 entries), an archetype library (24 professional-discipline archetypes used as lenses on the work), a release gate (`tools/check_release_readiness.py`), a doc-audit ratchet (`tools/doc_audit.py`), a token-timer grounding layer (`sadp/token_timer.py` — 646 lines), and a cross-session docket (`sadp/DOCKET.md`). Together these form a protocol that has run for roughly seventy sessions across approximately fifteen months on a single project: Acervator, a cryptocurrency trading bot system.

The protocol is not a methodology in the Agile/Scrum sense. It is closer to a recipe-book of cascade rituals — what you must do when shipping a change, what evidence you must capture, what structural tests must remain green, and how you must record what you learned so that the next session of the AI pair programmer can pick up cleanly. Most of the rules were born from a specific failure: a thing broke, the failure was logged, a rule was written to prevent the failure class. The rule count is therefore a rough proxy for the surface area of distinct failure modes encountered.

Who SADP is for

Software practitioners who are building substantial systems with an AI pair-programmer (Claude, GPT-class models, or future equivalents) over a timescale of weeks to years rather than a single sitting, and who notice the recurring frictions: the AI forgets context across sessions, makes the same mistake twice, asserts confidence it cannot back up, drifts from agreed direction, fails to verify before declaring done. SADP is the answer one practitioner — operating on one substantial project — developed for those frictions. The rules and rituals here may or may not transfer to other contexts. Section 3 is honest about what the framework claim does and does not yet justify.

What SADP is not

- **Not a finished methodology.** SADP is at version 1.92 and has been iterated approximately fifteen times in a single calendar quarter. Rules have been added, modified, and deprecated. The shape is still moving.
- **Not validated against a second project.** The protocol has been exercised on exactly one codebase. Whether it survives transplant to a second project is an open empirical question. See section 3.
- **Not a replacement for software engineering discipline.** SADP assumes the practitioner already understands testing, version control, code review, and incremental delivery. SADP adds rituals on top of those fundamentals; it does not substitute for them.
- **Not a model-control framework.** SADP does not constrain what the AI model can or cannot do at inference time. It structures the work surrounding the model: what you ask, what you record, what you check before you ship.
- **Not a productivity-multiplier claim.** SADP does not claim it makes AI-assisted development faster. It claims it makes AI-assisted development auditable over long timescales. The two are different.

The N=1 reference implementation

Throughout this document, the Acervator cryptocurrency trading bot is referred to as the N=1 reference implementation. This is the single project on which SADP has been exercised. It is a Python/PySide6 application with a substantial test suite (3,764 tests as of v3.22.6), an event-sourced trade ledger, and a multi-bot orchestration layer. The size of the codebase and the length of the development arc are large enough to have surfaced a substantial number of distinct failure modes — which is why the rule count is seventy-seven and not seven. But it is still one project, in one language, with one practitioner. The second N is not yet attempted.

2. Framework genesis

Session 22 — the moment that demanded a protocol

SADP was not designed up front. It was extracted from a specific failure pattern that surfaced in Session 22 of Acervator development. The pattern was: a regression that had been claimed-fixed in Session 19 had reappeared, the AI pair programmer had no memory of the prior fix attempt, and the human operator had no compact way to convey the history. The fix-and-forget loop had run three times without anyone noticing it was the same loop. The cost was several hours of repeated diagnosis on a problem that had already been diagnosed twice before.

The operator's response was a question: what would make this loop visible the third time it happened? The answer that emerged: an append-only log of what had been done, indexed by the rule it implicated, with the AI pair programmer expected to re-read the log on cold-start of every session. That log became `EDIT_LOG.jsonl`. The first three rules — R1 lock-state default, R3 append-only log, R4 episodic-memory MEM-N notation — emerged in Sessions 22 through 24 as the minimum machinery to make the fix-and-forget loop visible.

Sessions 23–25 — the anti-pattern audit

Once the first three rules were in place, an audit of the preceding twenty-two sessions surfaced a recurring catalogue of anti-patterns: bare except clauses swallowing errors, confidence claims with no code citation, banner version bumps without the test count moving, postmortems duplicating each other, departmental review claims not being re-verified across sessions, hops opening with optimistic summaries that did not match the code. Each anti-pattern became one rule. By the end of Session 25, the rule count had grown from three to sixty-five, and the protocol had a name: SADP, the Structured AI Development Protocol. Version stamped 1.0.

Sessions 25–27 — session-anti-pattern lessons

The next two sessions surfaced a different category of failure: lessons learned within a session were not being captured in a form the next session could find. The fix was R66 anti-pattern audit discipline and R67 session-anti-pattern lesson capture: every session must close with a written audit of what was learned about how to avoid the failure modes encountered, filed in a way the next session's cold-read protocol will see. The protocol versioned to 1.20 at this point and stabilized for a brief period.

Sessions 38–52 — first principles and observability

A second wave of rules surfaced during a long arc of fixing trading-engine bugs. R68 First Principles Come First was born from a session in which three speculative fixes had been shipped for a bug whose root cause had not been audited; the rule says: when a bug resists multiple speculative fixes, STOP shipping fixes — audit the actual code state, add observability, let the system tell you what is true. R69 Cold-Read Protocol and R70 Recurring Context Notification followed. R70 in particular changed the rhythm of the protocol: from per-ritual-point reporting (predecessor R35) to per-turn reporting of the AI's context budget, so that the human operator can see when the AI is approaching context exhaustion in real time rather than discovering it after the session has degraded.

Sessions 53–70 — discipline maturation

The most recent wave of rules — R71 through R77 — addresses specific failure modes that surfaced as the system grew more complex: silent flag-driven behavior (R71 Observability), hardcoded constants overriding operator settings (R72 OTSSOT), ad-hoc state mutation (R73 State Management), gate-stack bypasses (R74 Safety), reference-point ambiguity (R75 PRD), feature wiring without runtime activation (R76 Wiring), and static-binding failures (R77 SBR). The protocol versioned to 1.92 with the addition of R77. Each rule is paired with at least one canonical-failure MEM citing the specific evidence that motivated it.

Why the version is 1.92 and not 2.0

SADP versioning follows a deliberate convention: minor increments per rule addition or material modification, no major increment until the framework demonstrates portability to a second N. Section 3 elaborates. The framework is not at 2.0 because it has not yet been exercised on a project other than Acervator. When and if a second project adopts SADP and the rules survive contact with a different codebase, the version may increment to 2.0. Until then, the framework is honestly labeled as a single-project artifact at version 1.x.

3. Prior art and the framework-claim acknowledgment

This section exists because of a structured archetype review (AI3 Research Engineer lens, captured in docs/audits/2026-06-06_ai3_review_of_sadp.md) that surfaced a critical observation: the SADP framework claim was being made stronger than the evidence supported. Specifically, calling SADP a 'framework' implies generality that has not yet been demonstrated. The honest framing is: SADP is a codified shape of one project's learnings, structured in a form that may or may not transfer. This section enumerates what already exists in the world — prior art — so that the reader can evaluate which SADP primitives are genuinely novel and which are restatements of existing ideas in a project-specific dialect.

Primitives and their prior art

The table below maps each SADP primitive to the closest prior art the AI3 review surfaced. Citations point to widely-known industry references; the SADP-specific contribution column names what (if anything) SADP adds beyond the prior art.

SADP primitive	Prior art	SADP-specific contribution
Append-only event log (EDIT_LOG)	Event sourcing (Fowler 2005); audit logs (every regulated industry); Git itself	Indexed by rule + session + cascade unit for cross-session AI-pair retrieval
Episodic memory store (MEM-N)	Architectural Decision Records (Nygard 2011); engineering notebooks (Babbage to present)	Numbered, schema-validated, rule-tagged, retag-by-rule queryable; purpose-built for AI cold-read
Rule taxonomy with canonical-failure cites	Postmortem culture (SRE Book, Beyer et al. 2016); root-cause analysis (Boeing/NASA)	Failure-mode-to-rule explicit mapping with state machine (LOCKED/UNLOCKED/EXPIRED)
Release gate (check_release_readiness.py)	CI/CD gates (every CI system); Continuous Delivery (Humble & Farley 2010)	Project-specific gate composition + banner-bump refusal without [OK] Release-ready
Cascade-atomicity protocol (R5)	Database ACID transactions; deployment runbooks; release checklists	Atomic at the documentation-state level, not just the deployment level
Doc-audit ratchet (R25 DARP)	Ratchet metrics in test-driven development (e.g., mutation testing); compiler warnings as errors	Documentation drift as a numerical ceiling that monotonically decreases; ship gate fails on regression
Structural test family (labeled-vs-derived-truth)	Property-based testing (QuickCheck, Hypothesis); contract testing (Pact)	Specific structural family that catches a category — labeled state diverging from derived truth — across wizard / session-files / banner / claims / concepts

SADP primitive	Prior art	SADP-specific contribution
Token Timer grounding layer	Periodic safety checks; airline checklists (Gawande 2009)	Token-budget grounding specific to AI-assisted dev; anti-degradation per session
Recurring Context Notification (R70 RCN)	Self-reporting status in distributed systems; pilot/co-pilot callout discipline	Per-turn context-budget reporting; operator early warning of context exhaustion
Cold-Read Protocol (R69)	Onboarding documentation; SRE runbooks; oncall handoff	Designed for AI pair-programmer cold-start; bounds orientation to ~2-3% context budget
Code-Evidence Mandate (R6 CEM)	Evidence-based medicine (Sackett 1996); academic citation discipline; root-cause analysis	Applied as turn-by-turn discipline on an AI pair-programmer's claims about a codebase
Multi-archetype review	Multiple-perspective design review (Brooks 1975); Six Thinking Hats (de Bono 1985); red team / blue team	Codified archetype library with explicit failure-mode 'bland praise is the failure mode'
Departmental Lead Rotation (R11 DLR)	Code review rotation; pair programming partner rotation	Specific to the persistent-lead model; rotates which archetype drives critique each cycle
First Principles Come First (R68)	Toyota 5-Whys; root-cause analysis; Feynman troubleshooting	Triggered at three-speculative-fix threshold; mandates observability before further fixes
Engine Parity Mandate (R57 EPM)	Production / staging parity (Twelve-Factor App, item X); shadow deployments	Production engine ↔ simulation engine cascade-gate refusal; specific to dual-engine architectures

Where SADP is genuinely contributing something

Once prior art is acknowledged, the genuinely-novel contributions of SADP appear to cluster in two areas. First: the combination of an AI-pair-programmer-cold-read-optimized memory layer with a structural test family that catches label-vs-derived-truth divergence. The combination matters: the memory layer ensures continuity, the structural tests ensure the continuity is actually true rather than merely asserted. Second: the cascade-atomicity protocol applied at the documentation-state level, treating release notes, CHANGELOG entries, and MEM additions as one transactional unit with the code change. Most release engineering treats these as separate concerns; SADP forces them to ship or fail together.

What SADP does NOT yet demonstrate

- **Transferability.** No second project has adopted SADP. Until one does, the framework claim is structurally premature.
- **Scalability across multiple practitioners.** SADP has been operated by a single human-operator + a single AI pair-programmer. Whether it survives team contexts (multiple humans, possibly multiple AI agents) is unknown.
- **Cross-language portability.** SADP has run on Python only. The release gate, the test discipline, the schema validation, and the doc-audit tool are all implemented in Python.
- **Cross-model portability.** SADP has run with Claude (specifically Opus-class models with 200k → 1M context). Whether the cold-read protocol, the RCN cadence, or the cascade-gate enforcement survive different model families is untested.
- **Long-tail durability beyond seventy sessions.** The protocol has run for ~70 sessions across ~15 months. Whether the rule-accretion rate is sustainable, or whether the rule set bloats into incomprehensibility past a certain size, is an open empirical question.
- **Cost-effectiveness.** SADP imposes substantial overhead per ship — cascade artifacts, MEM authoring, EDIT_LOG entries, release-gate runs, doc-audit ratchets. The protocol assumes this overhead is dominated by the cost of regressions it prevents. The N=1 evidence is consistent with this, but the counterfactual (Acervator-without-SADP) has not been run.

The honest framing

SADP is a single-project artifact at version 1.92. It contains seventy-seven rules, hundreds of memory entries, and a structural test family. It was built incrementally in response to specific failure modes encountered on Acervator. Whether it is a framework — meaning, whether it would survive transplant to a different project, a different language, a different model, or a different practitioner — is an empirical question that has not yet been answered. The current evidence supports the claim that SADP works for the Acervator project. The current evidence does NOT support the stronger claim that SADP works in general. This document is structured around that distinction.

4. Design principles and the seven invariants

Underneath the seventy-seven rules lie seven design principles — invariants the rules are collectively trying to preserve. The principles were not written down up front; they were extracted after the fact from the pattern the rules exhibited. Naming them makes it easier to evaluate proposed new rules: a new rule that does not preserve at least one of the seven invariants is probably out of scope for SADP.

Invariant I — Every claim is backed by code

When the AI pair programmer states that a thing is true about the codebase, the statement must be accompanied by a citation (file path + line number, or test name + assertion, or MEM-N reference). Unbacked claims are the primary source of drift over long sessions. R6 CEM is the explicit rule; R28 DCA extends it to documentation; R69 cold-read protocol extends it to session start.

Invariant II — Cascades are atomic

A code change, a CHANGELOG entry, a MEM addition, an EDIT_LOG row, a banner bump, and a release-gate run are one unit. They ship or fail together. R5 cascade atomicity is the rule. The cost of this discipline is real (every ship requires the full cascade) and the benefit is real (the documentation state never diverges from the code state).

Invariant III — The release gate is sovereign

No banner version bump is permitted without [OK] Release-ready from tools/check_release_readiness.py. R2 PRM is the rule. Operator directive 2026-05-31, captured verbatim: 'FIX IT AND NEVER DO THIS AGAIN.' The directive followed a ship in which banners were bumped while the suite had eighteen failures. Since R2 was hardened, no banner-bump-without-gate failure has recurred.

Invariant IV — Memory is queryable, schema-validated, and append-only

EDIT_LOG.jsonl is append-only. EPISODIC_MEMORY.json is schema-validated. Rules are tagged on every MEM entry and every EDIT_LOG row. Queries like 'show me every MEM that implicated R57' are O(n) scans on the JSON file. The schema is enforced by tests (R20 SVG). The structure is therefore stable enough to support tools — manual_topic_matrix.py, doc_audit.py, qa_status.py — that all consume the memory layer.

Invariant V — Continuity survives session boundaries

Every session opens with a cold-read protocol (R69) bounded to ~2-3% of the AI's context budget. The cold-read does not full-read large files; it queries the orientation layer (hop_open.py, the MEMORY.md auto-memory, the DOCKET) and loads only what the upcoming task needs. R12 HC and R69 are the rules. The discipline is verified by the rate of operator-correction events in opening turns — when it drops, the cold-read is working.

Invariant VI — Structural tests catch labels-vs-truth divergence

A specific class of bug surfaces repeatedly: a label or a state flag asserts something the underlying derivation does not actually support. Wizard flags claiming pages are valid when they are not. Session-integrity files claiming a hop closed cleanly when it did not. Banner triples disagreeing across three locations. Claims audit triggering on language the evidence does not support. Concept-novelty counters asserting newness on restated ideas. SADP has a structural test family (MEM-428, MEM-429, MEM-430, MEM-433, MEM-445, MEM-446) that catches each of these. The family is described in detail in section 9 (v3.22.8).

Invariant VII — The protocol must be honestly framed

When the protocol overstates its evidence, the protocol itself is undermined. SADP therefore contains a claims-audit tool (sadb/_tools/claims_audit.py) that scans documentation for puffery — superlatives, unfalsifiable virtue claims, comparative claims without a baseline. Ceilings are enforced as ratchets. The Empirical Discipline Notice in Part 1 of the Acervator manual codifies this for the manual; this section codifies it for SADP itself. Section 3 of THIS document is the most important expression of Invariant VII: SADP is honest about what it has not yet demonstrated.

5. Rule taxonomy

Tier — core versus project-overlay

Every rule is tagged either **core** or **project_overlay**. A core rule is one that is expected to apply to any project that adopts SADP — invariants like cascade atomicity (R5), release gate sovereignty (R2), code-evidence mandate (R6). A project-overlay rule is one that is specific to the Acervator project — a particular wizard structure (R36 WPOD), a particular GUI widget layout (R46 GLCLR), a particular trading-engine structural detail (R57 EPM). The split matters for transferability: if SADP is ever transplanted to a second project, the core rules are the candidates that should ride along, while the project-overlay rules likely need replacement with the second project's equivalents.

Tier	Rule count	Meaning
core	60	Candidate for transplant to a second SADP project
project_overlay	17	Specific to Acervator; likely needs replacement on transplant

Category — what the rule governs

Rules R68 and later are also tagged with a category field describing the domain of behavior they govern: Process, Observability, Configuration, State Management, Safety, Reference Selection, Wiring, Static Discipline. The category field is younger than the rule corpus and is therefore present only on the more recent rules. Older rules can be retroactively categorized but the schema migration is not yet complete.

Lock-state — LOCKED / UNLOCKED / EXPIRED

Every rule has a state. **LOCKED** is the default — the rule is in force and must be honored. **UNLOCKED** is a deliberately-relaxed state, used rarely, for which the rule is currently suspended (typically because a known short-term exception is being managed). **EXPIRED** is the state for a rule that has been deprecated — the rule is preserved in the registry for historical traceability but is no longer enforced. No SADP rule has been EXPIRED as of v1.92; every rule introduced has been retained.

Dependency graph

Rules can declare dependencies on other rules via the `depends_on` field. The dependency graph is currently sparse — most rules are independent — but a few clusters exist: the cascade-atomicity cluster (R5 + R28 + R7 banner triple), the memory cluster (R3 + R4 + R20 schema validation), the release-gate cluster (R2 + R8 test count pin + R9 suite health + R21 coverage floor). The graph is rendered in the dependency-graph appendix in v3.22.8.

6. Every rule R1 through R77

Each rule entry below names the rule, gives its tier and category, summarizes the failure mode it prevents, and cites the canonical-failure MEM(s) that motivated it. Where a description-of-record exists in the registry, that text is reproduced verbatim. Where only a short title exists (pre-schema rules), a one-sentence summary derived from the rule's operational pattern is provided. Every rule is currently in LOCKED state unless noted.

R1 — Universal Lock-State Default

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Canonical-failure MEMs: MEM-001, MEM-004, MEM-054, MEM-066

R2 — Pre-Release Mandate (PRM) — release gate

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R3 — Append-Only EDIT_LOG

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R4 — Episodic Memory persistence (MEM-N notation)

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R5 — Cascade Atomicity — finalize as one unit

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Canonical-failure MEMs: MEM-001, MEM-010

R6 — Code-Evidence Mandate (CEM) — no claim without code citation

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R22

Canonical-failure MEMs: MEM-002, MEM-003, MEM-005, MEM-010

R7 — Banner Triple Mandate (BTM)

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R8 — Test Count Pin — drift detection

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R9 — Suite Health Gate — no skipped/xfailed without ADR

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R10 — Patent-Flag Discipline (PFD)

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at

the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Canonical-failure MEMs: MEM-012

R11 — Departmental Lead Rotation (DLR)

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R12 — Hop Continuity (HC) — session handoff via HOP file

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R13 — Operator Voice Preservation (OVP)

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R14

R14 — Cross-Reference Closure (CRC)

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R20, R22

R15 — Cooldown-Aware Retry (CAR)

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R16 — Zip-Pair Discipline (ZPD) — sibling + self-contained

tier: project_overlay

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R17 — ADR Index Maintenance (AIM)

tier: core

Investigation complete, rule re-locked

Depends on: R18

R18 — Chronicle Maintenance (CM) — Developer Chronicle continuity

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R19

R19 — Wizard-State Synchronicity (WSS)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R20 — Schema-Validated Governance (SVG)

tier: core

Restored from SUSPENDED by admin.

Depends on: R22

R21 — Coverage-Floor Ratchet (CFR)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R24

Canonical-failure MEMs: MEM-053

R22 — Mutation-Score Floor (MSF)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R20

Canonical-failure MEMs: MEM-005, MEM-011, MEM-012, MEM-066, MEM-092

R23 — Mypy / Bandit / pip-audit baseline (MBP)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Canonical-failure MEMs: MEM-014

R24 — Vulture Dead-Code Ceiling (VDC)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R25 — Doc-Audit Ratchet Protocol (DARP) — entry/exit

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R6, R22, R28, R37

R26 — Close-of-Session Discipline (CSD)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at

the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R35

R27 — Postmortem Rotation Cap (PRC)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R21, R24, R26

Canonical-failure MEMs: MEM-006, MEM-007, MEM-014

R28 — Documentation-as-Cascade Artifact (DCA)

tier: core

Placeholder entry filled in during Session 18 / v3.9.15 technical debt cleanup (MEM-121). R28 was referenced from RAIntSimBat code tags and other rule docstrings as early as Hop 3, but the rule text itself was never written — only the identifier was reserved. This writeup crystallizes the doctrine that was already in informal use.

Canonical-failure MEMs: MEM-002, MEM-003, MEM-004, MEM-005, MEM-006, MEM-007, MEM-008, MEM-009 (+8 more)

R29 — Stress-Test Battery Discipline (STBD)

tier: core

Placeholder entry filled in during Session 18 / v3.9.15 technical debt cleanup (MEM-121). Like R28, R29 was referenced from code tags (RAIntSimBat line 934 `# sadp: R28 R29 # microstructure slippage helpers`) but the rule text was never written.

Depends on: R28

R30 — Property-Test Coverage Floor (PTCF)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Canonical-failure MEMs: MEM-013

R31 — Indicator Coverage Pin (ICP)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R32 — Bare-Except Forbiddance (BEF)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R28

R33 — Architecture Fitness Function (AFF)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Depends on: R29

R34 — Hypothesis Property Strategy Sharing (HPSS)

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

R35 — Ritual-Point Reporting (RPR) — predecessor of R70 RCN

tier: core

Pre-schema rule. Operational summary: see legacy title above. Schema migration to add description-of-record is queued in the post-manual-arc cleanup. Authoritative behavior is documented at the rule name's first appearance in sadp/EDIT_LOG.jsonl and in the MEM(s) cited below.

Canonical-failure MEMs: MEM-014

R36 — Wizard Page-Order Discipline (WPOD)

tier: core

R36 added — FTP (Follow the Procedure) user-invoked compliance gate.

Depends on: R25, R35

Canonical-failure MEMs: MEM-092

R37 — Settings-Tab Parity (STP)

tier: core

R37 added — Uncertainty Declaration rule.

R38 — Spendable-Widget Layout Stability (SWLS)

tier: core

R38 added — SADP Annotations on Critical Paths.

Depends on: R25, R36

R39 — L8 OOS Attribution Pin (L8AP)

tier: core

R39 live — Atomic File Migration. SADP v1.3 portability implemented.

Depends on: R27, R37

R40 — Pattern-Extension Audit (PEA)

tier: core

R40 added — LTM Episodic Memory Query procedure.

Depends on: R26, R35

R41 — Phantom-Timeframe Discipline (PTD)

tier: core

R41 added — GUI Widget Initialization Scope Safety.

Depends on: R6, R28

Canonical-failure MEMs: MEM-051, MEM-052, MEM-057, MEM-070, MEM-071

R42 — Cross-Engine Sync — Simulator ↔ Battery

tier: core

Any change to accumulation engine logic in `simulator.py` (`_sim_scrumming_tick`) OR `RAIntSimBat.py` (`run_v3192`) MUST be mirrored in the other file in the same session. The two engines are parallel implementations of the same strategy. Divergence is a silent regression — the battery tests a different bot than the one the user sees.

Depends on: R6, R22

Canonical-failure MEMs: MEM-066, MEM-095

R43 — Fee Guard — No Trade If Profit $\leq$ Fee

tier: core

No trading platform adjustment may generate a transaction that results in a loss due to trading fees. If the computed profit margin for a trade is less than or equal to the fee cost of that trade, the trade MUST NOT execute. Period. No exceptions. Applied to: FOLD, BULLSEYE_FOLD, BOOST_FOLD, REH_DUMP, SHAD_DUMP, DIST. For accumulation entries (SCRUM, REH_ENTRY, SHAD_ENTRY) where profit is future-dependent, apply a minimum expected gain check: price must be at least `fee_rate` below the reference price before entry is allowed.

Depends on: R28

R44 — Slop Code Prevention — Elegance and Tightness

tier: core · state: **ACTIVE**

Code must be as elegant and tight as possible. Slop code — defined as unnecessary duplication, magic numbers, overlong functions, and inline re-definition of named concepts — is a defect, not a style preference. Specific requirements: (1) No magic numbers in engine logic — use named constants. (2) Any logic block repeated 2+ times must be extracted into a helper. (3) No inline dict/list re-created on every call that could be a class constant. (4) Defensive guard blocks use a loop, not N separate `hasattr()` lines. (5) Functions > 200 lines require a documented justification in a TECH_DEBT entry.

Canonical-failure MEMs: MEM-070, MEM-071, MEM-092, MEM-100

R45 — Verbatim Integrity for Routing Identifiers (VIRI) — No Guess, No Propose

tier: core

When handling data that routes real-world value — crypto wallet addresses (BTC, ETH, L2s, any chain), bank account/routing numbers, SWIFT/IBAN/BIC codes, payment URLs with amount parameters, private keys, API keys routing funds — the AI MUST NOT modify the value based on inferred intent, pattern matching, checksum auto-correction, typo heuristics, or any form of guess. Additionally, the AI MUST NOT propose a specific corrected form to the user. Proposing 'I think you meant X' creates a confirmation-bias attack surface: a user saying 'yes' by reflex locks in a guess that may be catastrophically wrong. REQUIRED BEHAVIOR when a routing identifier appears malformed: (1) preserve the identifier verbatim in every written output; (2) describe the observation objectively — what the AI sees, what the standard

format is, and the specific deviation — WITHOUT proposing a specific replacement; (3) request that the user re-provide the identifier by typing it explicitly; (4) use ONLY the explicit user-provided replacement in any subsequent output. SCOPE: applies to all identifiers where small textual differences produce other valid, non-error routing targets and where the consequence of a mistake is irreversible (funds lost, credentials compromised). Does NOT apply to identifiers where errors surface as 'not found' failures the user can recover from (URLs to public pages, email addresses for non-financial correspondence, syntactically impossible identifiers like 5-char ETH addresses where any tooling will reject them before value moves).

Depends on: R25, R28, R37

R46 — Mandatory LTM Query Before Historical-Concept Implementation (MLHCI)

tier: core

When an AI session is asked to implement, port, fix, or verify any feature that references a named historical concept — i.e. any concept whose canonical specification was written in a prior session — the AI MUST execute an LTM query (R40) against the episodic memory BEFORE writing any code, specifying any design, or proposing any interpretation. TRIGGERS (non-exhaustive): (1) A named concept appears in code comments, CHANGELOG, or conversation history but has no complete specification in the current context window. (2) The user references a feature by a capitalised name ('Nuclear Mode', 'INFERNO', 'Charge-Up', 'Shadow Balance', 'Hedge', 'REH', 'Smart Wire', 'Harvest-Fold Cycle', etc.). (3) An in-code identifier (state name, variable, class) has a name implying intent ('_scrum_target_mode', '_delta_flash_state', '_reh_queue_ok') that the AI cannot trace to a complete in-window definition. (4) The conversation references 'origin point', 'canonical', 'the spec', 'how it was designed', or similar. PROHIBITED BEHAVIOURS: (1) Reading one or two lines of implementation code and inferring the complete specification from them. (2) Proposing a specification ('I'll implement X with threshold Y') without first confirming that Y matches the historical spec (or noting that no historical spec exists). (3) Shipping a feature whose name matches a MEM-entry topic without citing the MEM-id in the code comment or commit log. REQUIRED BEHAVIOUR: STEP 1 — LTM QUERY: search the episodic memory for every distinctive term in the feature name. Use R40 scoring (title=3, keyword=2, summary=1, rule=4, recency bonus). STEP 2 — SCORE THRESHOLD: if the top-hit score is ≥ 5.0 , that entry is authoritative and its spec governs the implementation. If the top-hit score is < 5.0 , note that no authoritative spec was found and proceed under R37 (propose-and-confirm) with explicit disclosure. STEP 3 — CITE IN CODE: any code implementing a historical concept must reference the source MEM-id in a comment ('sadp: R40, see MEM-081'). This makes the memory traceable from the implementation forward, not just backward. STEP 4 — MIRROR IN DOCS: any CHANGELOG, HOP log, or memory entry describing the work must cite the MEM-id it was sourced from. SCOPE BOUNDARY: R46 applies to SEMANTIC specifications (what a feature does, what its thresholds are, what its visual palette is). It does NOT apply to: (a) Pure bug fixes where the intended behaviour is self-evident from context. (b) Syntactic refactors (renames, extractions) that preserve behaviour. (c) New features with no historical antecedent.

Depends on: R36, R40

Canonical-failure MEMs: MEM-092

R47 — File and Approved Changes Protections (FACP)

tier: core

Once a change has been implemented successfully and approved, the affected code becomes LOCKED against further modification by any Agent. Lock protects work product from re-specification drift. APPROVAL SOURCES (any ONE triggers lock): - Explicit Board approval (human user): 'ship it', 'yes', 'looks good', or equivalent affirmation. - Successful canonical verification (battery pass, test suite green). - Completion of an earmarked task per session handoff document. - Presence in the pristine baseline source at session start (inherited lock from prior Hop). UNLOCK AUTHORITY: DEPARTMENT LEAD. Only a Department Lead can approve an unlock request. Agents and Supervisors cannot self-unlock. Project Lead and Board may override a Department Lead decision (escalation direction only — Agents may not appeal over a Department Lead's head without going through the Supervisor chain). In single-instance sessions where no separate Department Lead exists, unlock requests escalate directly to the Board (human user) — the Agent does NOT self-promote to Department Lead for the purpose of self-approval. CRITICAL CATEGORIES — AUTO-UNLOCK PERMITTED (Board-approved): The following categories allow an Agent to proceed without prior Department Lead approval. The Agent must log the unlock at the moment of action, not session close. Department Lead reviews the log and may retroactively revoke/roll back if the categorisation was wrong. (a) SECURITY — confirmed vulnerability (CVE, credential leak, injection, privilege escalation). Not 'seems risky.' (b) SLOP REDUCTION — per R44, when slop materially degrades correctness or performance. Taste preferences do not qualify. (c) PRESIDING RULE COMPLIANCE — when a LOCKED/CORE rule (R1-R46) directly requires the modification and the Agent can cite the rule ID inline. PROHIBITED — REQUIRE DEPARTMENT LEAD UNLOCK IN ADVANCE: - 'I think I know better' modifications to approved work. - Re-specifying an approved design because in-window reading seems ambiguous → this triggers R46 (fetch memory), not R47 unlock. - Scope extension: 'while I'm here, I'll also change X.' - Treating a bug report as authorisation to redesign — a REPORT initiates DIAGNOSIS, not MODIFICATION. Only a scoped unlock initiates modification. - Agent self-categorising routine work as CRITICAL to skip approval. Department Lead review of auto-unlock logs MUST catch this or the rule fails. REQUIRED WORKFLOW (non-CRITICAL work on locked files): 1. Agent declares: 'R47 unlock requested for , scope , reason .' 2. Agent waits for Department Lead confirmation — no edit without confirmation. 3. Edits stay strictly within declared scope. No drift. 4. On completion, Agent logs the unlock-edit-relock cycle in session log + MEM entry. 5. Re-verification (test/battery) before relock. REQUIRED WORKFLOW (CRITICAL auto-unlock): 1. Agent declares category inline ('R47 AUTO-UNLOCK: SECURITY ') at the moment of first edit. 2. Agent performs minimum viable fix — no scope extension. 3. Agent logs in session summary for Department Lead review. 4. Department Lead reviews post-hoc; may revoke.

Depends on: R22, R36, R44, R46

Canonical-failure MEMs: MEM-092, MEM-095, MEM-099, MEM-100, MEM-102, MEM-103, MEM-104

R48 — Triple-Radius LTM Search on Negative Reports (TRLS)

tier: core

When a Board/user report indicates that a previously-shipped feature is broken, wrong, incomplete, or regressed, the Agent **MUST** execute a three-radius LTM + ecosystem search **BEFORE** proposing or implementing any fix. Single-keyword LTM queries miss the substrate mechanics and design-intent comments where the actual answer frequently lives. **TRIGGERS** (any one): (1) User reports a previously-shipped feature as broken, wrong, or not working. (2) User reports an earmarked prior-hop task as incomplete. (3) User reports a recent edit broke something that worked. (4) User uses escalation language: 'still wrong', 'still broken', 'same as last time', 'WTAF', or equivalent — an explicit signal that prior response missed something. **THREE RADII (ALL REQUIRED, IN ORDER)**: 1σ — **DIRECT NAME QUERY**: standard R40 scoring on the feature name. Capture top 10 hits. Typically surfaces the single most-named entry (e.g. MEM-081 for 'Nuclear Mode'). 2σ — **SUBSTRATE QUERY**: for each 1σ hit with score ≥ 5.0 , extract dependency terms from its summary and `related_entries`. Requery with those terms. Captures the mechanisms the feature depends on or visualises (e.g. MEM-063 charge-up permission gate, which MEM-081's charge-up visual wraps). 3σ — **ECOSYSTEM SWEEP**: full-text grep across: - Inline comments in files implementing the feature - Function docstrings in the implementing module - Pristine baseline source (if available in uploads) - All ACERVATOR_HOP*.md (not just current hop) - CHANGELOG.md full history - SPEC.md / SPEC-CORE.md - AI_DEVELOPER_GUIDE.md - DEVELOPMENT_CHRONICLE.md Design-intent often lives in comments and docstrings that were never memorialised into LTM (e.g. the 'After harvest: stay in TRACK mode for consecutive harvests' comment at simulator.py line 5862 that stated the Nuclear Mode fix verbatim). **REQUIRED LOG FORMAT** (before any fix proposal): R48 TRLS — Triple-radius complete: 1σ hits: [MEM-xxx (score N.N), ...] 2σ hits: [MEM-yyy (via 'dep_term'), ...] 3σ findings: [file.py:LLL "comment text", ...] Intent synthesis: [one-sentence, citing sources] Only **AFTER** this log is produced may the Agent propose a fix scope. The log is submitted to the Department Lead (or escalated to Board if no DL present) for R47 unlock approval. **PROHIBITED**: - Skipping any radius under time pressure or user urgency. - Declaring 2σ done without examining dependency terms + `related_entries` of top 1σ hit. - Declaring 3σ done without at least grepping the implementing file for design-intent comments. - Proposing a fix before the TRLS log is complete.

Depends on: R40, R46, R47

Canonical-failure MEMs: MEM-092, MEM-095, MEM-099, MEM-100, MEM-102, MEM-103, MEM-104

R49 — Historical Baseline Framing (HBF)

tier: core

Formalises the practice used continuously through sessions 17-18. Forensic edit trail is necessary when memory fails or git history is insufficient (e.g. multi-file edits in a single conceptual change, authority citations, rule compliance proof).

Depends on: R47

Canonical-failure MEMs: MEM-095, MEM-099, MEM-100, MEM-102, MEM-103, MEM-104

R50 — Doc-Audit Tool Authority (DATA)

tier: core

Session 18 exposed the 'memory gap paradox': the rule telling the agent when to consult rules is itself out-of-context. R48 TRLS existed through 8 Nuclear corrections and fired only AFTER user push-back, never proactively. More rules cannot fix this — the fix is structural: a guaranteed in-context anchor at session start. ACTIVE_ANCHORS.md is small enough that reading it is cheaper than the cost of NOT reading it. Deliberately anti-prescriptive: it does not say 'remember to apply R48' — it surfaces R48 with its canonical failures pre-linked, so landing on the anchor already primes the graph.

Depends on: R48, R49

Canonical-failure MEMs: MEM-094, MEM-097, MEM-098, MEM-099, MEM-100, MEM-102, MEM-103, MEM-104

R51 — Batch A — Features / Chronicle / ADR Ratchet (BACFAR)

tier: core

Session 18 exposed the write-time counterpart to the read-time memory-gap paradox. R50 ensures the agent reads the graph at session start. R51 ensures the agent writes the graph at edit end. Without R51, 25 MEMs accumulated in session 18 without a single version bump: CHANGELOG lied about what was in the code (top entry [3.9.1] while `__version__` still read 3.9.0), Hop 4 doc was 5+ hours stale, `version_sweep.py` never ran, all audit findings were hidden. Board diagnosis (MEM-107): 'version numbers are not being iterated nor has any documentation been updated nor have any code base audits been ran for a while because v# has not bumped thus triggering the relevant cascade.' The version bump IS the forcing function. Skip it and the whole downstream cascade silently skips with it.

Depends on: R49, R50

Canonical-failure MEMs: MEM-105, MEM-106, MEM-107, MEM-108

R52 — Batch B — Rules Registry Ratchet (BRRR)

tier: core

Session 18 exposed a third failure mode in the memory-gap paradox beyond R50 (read-time) and R51 (write-time): mid-session backlog drop. Agent would respond to a new request with analysis + proposals, silently abandoning work explicitly listed as incomplete in the prior turn. Board caught this directly: 'do not forget the work from above' (first time); 'But...still...do not forget the work from above...lol' (second time). R52 was demonstrated effective through Board reinforcement before formal registration — itself a canonical case for the meta-clause.

Depends on: R50, R51

Canonical-failure MEMs: MEM-109, MEM-110

R53 — Coverage Wave Discipline (CWD) — 0% module triage

tier: core

Board directive 2026-04-20: 'Optimize further. Optimize all code. Add optimization as a required component of all audits.' Context: comprehensive RAIntSimBat run choked at final packaging step despite

successful compute phase. The compute finished cleanly (20 portfolios, 23 PDFs on disk); the tail assembly exceeded tool-call budget because packaging had no measured performance envelope and redid work the run_all.py driver had already done. R53 formalizes the discipline to catch this class of regression at every cascade going forward: did you measure what you changed?

Depends on: R48, R49, R51

Canonical-failure MEMs: MEM-113

R54 — Design-Coherence Refresh (DCR)

tier: core

Board directive 2026-04-20 after cumulative dissatisfaction with chart output: 'Charts are quite sloppy. Need some formatting and styling rules. Import relevant research. Augment SADP. I have seen consistent challenges with this broader issue and how you go about generating documents.' Self-audit of RAIntSimBat/reports_pdf.py found 30 scattered fontsize literals, 21 ad-hoc transAxes placements, 3 hex color literals at module level, symlog used to compress a \$77B outlier, tables that silently clipped at page boundaries via `if y < 0.05: break`, text truncated with [:60] / [:50]. Symptoms of no design system. R54 installs the discipline structurally so every future document renders against tokens, not judgment.

Depends on: R49, R51, R53

Canonical-failure MEMs: MEM-114

R55 — Project-Overlay Lock-State (POLS)

tier: project_overlay

Board directive 2026-04-20 (Session 18 Turn 11): 'RAIntSimBat - Slippage Protection - Wrap within the Search, Track, Fire sequence a period of Ultra Sampling until a full safe fill is verified completed. If lag in the fill is detected, adjust price up to the minimum profit threshold of 2% and cancel all else if further drift from the given Scrum or Fold occurs. This mechanism is a Verify Hit. The sniper is down scope until...pop.' Smoke test on BUFFETT × Apr24-Apr25 confirmed the mechanism is working: VH ON advantage \$29,828 vs VH OFF \$18,319 (+\$11,509, +63%). Runtime overhead only +10% (13.3s vs 12.0s). Cancel rates high (e.g. MSFT 7650 canceled vs 1 clean) — that is the point. The 2% profit discipline refuses marginal trades that would slip into losses.

Depends on: R28, R29, R42, R49, R51, R53

Canonical-failure MEMs: MEM-115, MEM-118

R56 — RAIntSimBat Elevation Audit (RSEA)

tier: core

Board directive 2026-04-20 after MEM-117 A/B regression finding: 'Every new trading-discipline rule requires an A/B comprehensive run, not just a smoke test, before registration locks.' R55 VH was registered on BUFFETT smoke test alone (4 sims, low-volatility selection bias). Shipped with wrong

calibration (entry_price reference). A/B later revealed -\$55.3B regression across 6 black-eye portfolios with 43 regressed vs 1 flipped. R56 ABV would have caught this before R55 registration locked. R56 ABV is the fix: make A/B validation a structural prerequisite for any trading-discipline rule, not an afterthought.

Depends on: R49, R51, R52, R53, R55

Canonical-failure MEMs: MEM-117, MEM-118

R57 — Engine Parity Mandate (EPM) — production↔sim cascade gate

tier: core

Board directive 2026-04-20: 'Validated RAIntSimBat updates should be getting pushed and verified as present / functional in the main engine / governance layer.' R48 TRLS investigation of src/core/nuclear_live.py found the _tick_feed function executing scrum and fold trades with naive fills: no microstructure slippage (R28/R29), no Verify Hit (R55), no confidence gates, no provenance folding (R42). The live engine had drifted significantly from RAIntSimBat's execution discipline. R55 VH (just born MEM-115/v2 MEM-118) was in the sim but absent from the live path — meaning real/paper trades would execute without the slippage protection R55 was designed to enforce. R57 EPM is the fix: make parity a structural prerequisite. Every sim-validated execution rule must show up in the live engine with functional parity, verified by test, recorded in RULE_REGISTRY. Closes the sim-to-live gap for this class of rule.

Depends on: R49, R51, R52, R53, R55, R56

Canonical-failure MEMs: MEM-119

R58 — Capital Registry USD-Anchoring (CRUA)

tier: project_overlay · state: **ACTIVE**

Group FF PoA Tournament v1 (MEM-128) introduces forked-state play above the production bot stack. Without TBI enforced as a first-class rule, future tournament features (elimination damage models, monster interactions) could accidentally write back to production state. Born before that can happen.

Depends on: R42, R43, R44, R57

R59 — Inverted Extractor Direction Flag (IEDF)

tier: core · state: **RESERVED**

User directive: 'when new features are added or modify, require a hashed validation prompt from within the software to mark the task as completed. Smartly update Nuclear mode for this and all future SADP projects to integrate stress testing (test points) within the code base.' Turn 1 of a 6-turn hop plan: core module + tests + signing. Subsequent turns: layer_watch auto-insertion, Nuclear in-app prompts, R59 full birth + R56 amendment, legacy migration, tournament integration.

Depends on: R28, R29, R49, R56, R57

R60 — cp1252 Preventative (CP1252P) — UTF-8 stdout reconfigure

tier: core · state: **ACTIVE**

User directive 2026-04-20 Session 18 end-of-turn: 'SADP - New Rule - Older work pre-empts newer. Tasks get added to the back of the queue; not the front.' Formalizes a discipline that was already being applied informally (verify-first pattern, halt-mid-turn halt/continue, and repeated priority board additions at the bottom). User was self-applying this rule in the same message by writing 'Add to Docket - Finish Older Tasks First' for the bounty monster design. Rule formalized same turn it was requested because it governs the handling of the other items in the same message.

Depends on: R28, R47, R52

R61 — Per-Trade Log Granularity (PTLG)

tier: core · state: **ACTIVE**

Four documented regressions in Session 18 alone (MEM-125 Windows cp1252 crash mask, MEM-127 psutil silent mask, MEM-139 3-version blind UI from closure shadow, MEM-141 TestNet empty tables from refresh swallows) all traced to `except Exception: pass` at callback boundaries. Each was patched at ONE site; the pattern was never class-eliminated. R61 closes the class: no silent callbacks, no Swiss-cheese patching. Formalizes the discipline so future changes cannot reintroduce silent-swallow undetected.

Depends on: R28, R49, R52

R62 — Trade Grader Library Isolation (TGLI)

tier: core · state: **ACTIVE**

R26 was silently violated for 24 hours across all Session 18 cascades (40 addenda, 63 MEM entries, DEVELOPMENT_CHRONICLE.md never updated past April 19). Operator caught the drift during HOP prep review and was rightfully angry. Root cause: R26 exists in SPEC-CORE.md and is referenced in HOP4.md, but was not enforced at every cascade checkpoint. Rule-drift from omission cannot be prevented by exhortation alone; it requires a structural check that runs every cascade and blocks proceeding when drift is detected. R62 formalizes that structural check and names the integration point. Operator directive phrase: 'If some rules have to have their own short term memory loops so be it. We need alignment.' Design: each invariant gets one function; registry of FreshnessCheck entries; runner prints pass/fail report; exit code drives cascade/hop-open behavior.

Depends on: R26, R28, R49, R50, R51

R63 — Cooperative-Capacity Arbiter Authority (CCAA)

tier: core · state: **ACTIVE**

Operator directive Session 20 2026-04-21 verbatim: 'You never augment on your own. You go get resources as it is a resource itself. Not your sketchpad. Make it a rule.' External grounding: Wikipedia WP:V policy (en.wikipedia.org/wiki/Wikipedia:Verifiability) — threshold for inclusion is verifiability not truth,

material must be attributable to reliable published sources. Wikipedia WP:NOR policy (en.wikipedia.org/wiki/Wikipedia:No_original_research) — all material must be attributable to reliable published source; contributors don't publish original thought. Royal Society motto Nullius in verba (est. 1660) — evidence-grounded epistemics. IEEE 29148:2018 (already in SRS §1.4) — requirements traceability requires source attribution. R40 LTMQ / R46 MLHCI / R47 FACP / R49 MDEL — existing SADP lineage of 'check prior / log change / protect authority'; R63 extends from memory+file-level to doctrine-level. Session 19 empirical evidence MEM-149/MEM-154 — multiple self-authored SADP augmentations without external grounding (ADR-012, SRS rewrite, RISK_REGISTER additions, ARCHITECTURE rewrite); operator observed pattern and named it 'sketchpad.' R64 FCP (committed this session first chunk) — its own drafting was first explicit application of R63 principle: fetched Anthropic rate-limits docs, cited Helland, cited operator directive, cited Session 19 MEM-154. R64 draft is prototype pattern R63 codifies.

Depends on: R40, R46, R47, R49, R64

R64 — Bridge-Layer Contract (BLC) — Capital ↔ Arbiter

tier: core · state: **ACTIVE**

Operator directive Session 20 2026-04-21 verbatim: 'For long running, complex tasks implement forced chunking to avoid system chokes. Design ALL workflows to support. One file at a time. One page at a time. Whatever it takes. Task processing time **MUST** be reduced and we **MUST** stay within Anthropic's safety windows or this project is dead. I have not been able to make a single prompt.' External grounding: Anthropic Claude API Rate Limits documentation (platform.claude.com/docs/en/api/rate-limits) — token-weighted 5-hour rolling windows, weekly caps introduced Aug 2025, peak-hours tightening March 2026 (5am-11am Pacific weekdays) affecting ~7% of Pro users; operator is in Portland OR — directly in peak-hours zone. Anthropic operational guidance: 'The biggest rate limit drain is long conversations. Each message includes the full history. Start a new conversation for each distinct task instead of continuing one mega-thread.' Session 19 MEM-154 **NEGATIVE RESULTS** — P5 sweep consumed 2 full turns + hit tool-use ceiling mid-sweep + suffered compaction cliff. SE canon: Helland (2012) 'Life beyond Distributed Transactions' — atomic idempotent composable work units (same lineage as R28 FL / R29 IDM).

Depends on: R26, R52, R60

R65 — Geometric Design Grid (GDG)

tier: core · state: **ACTIVE**

Operator directive Session 20 2026-04-21: 'Augment SADP with GUI design rules.' Scope selection via interactive elicitation: all four sources chosen (Nielsen + WCAG + KDE HIG + Material Design 3). External grounding fetched this chunk: Nielsen Norman Group 10 Usability Heuristics (nngroup.com/articles/ten-usability-heuristics/) — 1994 with 2020 updates; W3C WCAG 2.2 (w3.org/TR/WCAG22/) — October 2023 release, 4 principles POUR, 13 guidelines, 78+ success criteria at A/AA/AAA; KDE HIG (develop.kde.org/hig/) — June 2024 rewrite by Nate Graham, Qt/Plasma-native desktop conventions; Google Material Design 3 (m3.material.io/) — current including May 2025 M3 Expressive update, 7 foundations + 3 pillars. Internal precedent: R17 GUI dimension discipline,

theme_engine.py cyberpunk dark theme, 30+ existing PySide6 GUI files as of v3.13.7. R28 FL and R61 CBF are code-level expressions of Nielsen heuristic #1 (visibility of system status) and #9 (recognize/diagnose/recover from errors) — the bridge from SADP governance to user-facing UX. First rule with multi-source external grounding under R63 ERG (committed Session 20 Addendum 2); first rule explicitly addressing GUI concerns in SADP.

Depends on: R17, R28, R61, R63, R64

R66 — Anti-Pattern Audit Discipline (APAD)

tier: core · state: **ACTIVE**

Operator directive Session 20 2026-04-21 Task 3 verbatim: 'HOP alignment is taking an EXTREMELY long time. Investigate all you did to achieve orientation, why, timestamping, find inefficiencies and squash them.' HOP audit Finding 1 measured this chunk (Session 20 chunk 7): script runs 414ms average (3 runs: 399/431/412ms); Claude's historical orientation pattern accumulates 3-8 tool calls before first response, extending wall-clock to 15-60s. R66 caps orientation at 1 tool call. External grounding: hop_open.py source design (MEM-151 R62 FRG birth) — deterministic by construction, output is canonical summary; R62 FRG structural gate; R64 FCP execution-grain discipline precedent (MEM-155); Anthropic operational guidance on long-conversation token bloat (platform.claude.com/docs/en/api/rate-limits, already fetched in R64 grounding) — operator in Portland in 5am-11am PT peak-hours tightening zone. Internal precedent R62, R64. Audit finding self-applied: this rule's own commit stays within R64 ceilings (3 tool calls: 1 bash cascade + 1 bash verify + 1 elicitation; 0 new output files; 0 rewrites).

Depends on: R62, R64

R67 — Session Anti-Pattern Lesson Capture (SAPLC)

tier: core · state: **ACTIVE**

Operator directive Session 23 2026-04-22 verbatim: 'SADP - Upgrade - Documented pathology awareness. We should have or enable greater awareness of known issues with the various libraries and resources we integrate into our codebase so as to avoid injecting what has been noted as "known pathologies" and Qt has flagged for several of them.' Triggering incident: MEM-204 (same session) — main GUI Console used QTextEdit for a log stream, froze on text-select, went sluggish after several hundred entries. Qt's own documentation at doc.qt.io/qt-6/qplaintextedit.html explicitly recommends QPlainTextEdit for 'displaying a large amount of text'. The pathology was vendor-documented and would have been avoided by a 30-second proactive check. External grounding: Qt 6 documentation on QPlainTextEdit vs QTextEdit performance tradeoffs; Python stdlib hash() nondeterminism doc (already cited in MEM-201); Coinbase Advanced Trade API granularity doc (already cited in MEM-203); three.js CDN version constraints from Anthropic's artifact prompt. Seeded registry sadp/KNOWN_PATHOLOGIES.md with 10 entries from this session + prior findings (Q-001 through Q-005, P-001, P-002, CB-001, CB-002, CCXT-001, CCXT-002, PY312-001). Integration: R63 ERG — DPA consultation IS external resource grounding for library-facing changes. R64 FCP — registry lookup is preflight, doesn't count against chunk budget. R52 BSP — registry itself is a BSP artifact (broad-sweep

pattern documentation).

Depends on: R26, R52, R63, R64

R68 — First Principles Come First

tier: core · category: Process · introduced: MEM-216 · session: 24

When a bug resists multiple speculative fixes, STOP shipping fixes. Audit the actual code state. Add observability. Let the system tell you what's wrong rather than telling the system what you think is wrong.

R69 — Capture New Domains as SADP Anchors

tier: core · category: Process · introduced: MEM-237 · session: 24

When a session pulls you into a domain you haven't worked in before (audio synthesis, shader programming, statistical method, protocol implementation, etc.), capture the learnings in a SADP-level knowledge anchor (e.g. AUDIO_CRAFT.md, sibling to VISUAL_CRAFT.md) BEFORE the session closes. Future sessions have no memory of what was learned unless it's written into SADP. Include: named authorities in the domain, 3-7 foundational pillars, Acervator-specific conventions adopted, concrete techniques that worked (with enough detail to reproduce), and a decision heuristic for when to go deeper (e.g. 'when should we add library X?'). Cite specific practitioners, not 'industry practice.' This is the VISUAL_CRAFT.md / DEPARTMENT_LEADS.md pattern.

Depends on: R50

R70 — Recurring Context Notification (RCN)

tier: core · category: Process · introduced: MEM-246 · session: 25

Extends R35 from ritual-point reporting (R25 Step 2, R26 close) to per-turn reporting. Claude emits a compact context-fill footer on every assistant response, format '█ctx ~N% · ~Nk/200k█' with escalation tags at 50/65/75/85/95% thresholds. No exceptions for short replies or tool-only turns. Tool outputs count toward fill. Round up when uncertain. If an operation will add >5% to fill, declare the cost first. See SPEC-CORE.md §R70 for the full escalation ladder.

Depends on: R35

Canonical-failure MEMs: MEM-246

R71 — Silent-Skip Surface (SSS)

tier: core · category: Observability · state: **active** · introduced: 3.15.81 · session: Session 26

Any code path that returns a per-item result with applied=False (or equivalent skip indicator) AND a machine-readable reason MUST also emit an operator-visible log on the same path. A silent skip — where the function captures the skip in its return value but no log fires — is a bug class even if the underlying logic is correct. The operator cannot diagnose what the code refuses to surface. Apply equally to dust-filter

skips, threshold rejections, missing-target skips, and per-item exception swallows. Throttling is permitted (e.g., once per N seconds) when the skip would otherwise spam; full silence is not.

Depends on: R28

Canonical-failure MEMs: v3.15.74 — SmartWireManager.distribute_fold_profit dust-skipped per-wire shares below min_wire_amount with no operator-facing log, v3.15.78 — LogManager.log_trade defined and writer constructed, but no caller invoked it; trade.log file silently empty for weeks

R72 — Operator-Tunable Sole Source of Truth (OTSSOT)

tier: core · category: Configuration · state: **active** · introduced: 3.15.81 · session: Session 26

When a feature is exposed to the operator as a tunable (spinbox, slider, percentage, threshold), the operator's value **MUST** be the **SOLE** source of truth for the gate it controls. Hardcoded constants, magic numbers, or secondary thresholds in the same code path that can **OVERRIDE** the operator's value violate this rule. Acceptable: the operator's value scales a hardcoded shape (e.g., position_boost percentages defined per-indicator). Unacceptable: a hardcoded threshold that fires regardless of the operator's setting on the same concept.

Canonical-failure MEMs: v3.15.75 — MEM-196 ripe-harvest used hardcoded 10% delta threshold and 0.80 BB position threshold that overrode operator's scrumming_interval_pct + scrum_detect_pct; operator-set values were decoration, not control

R73 — Runtime-Mutable State Needs Runtime-Mutable Checks (RMMC)

tier: core · category: State Management · state: **active** · introduced: 3.15.81 · session: Session 26

Any flag, threshold, or attribution decision based on a condition that the operator (or an event in another subsystem) can change at runtime **MUST** be re-evaluated on every decision point — not merely set once at initialization. Init-time evaluation is correct **ONLY** when the underlying condition is itself permanent for the bot/component lifetime. Examples of runtime-mutable conditions: 'does another bot share my target asset?', 'is the operator-set max_target_growth_pct currently > 0?', 'is the higher-TF phantom currently BULLISH?'. The minimum acceptable refresh cadence is the tick boundary; per-decision is preferred when the computation is cheap.

Canonical-failure MEMs: v3.15.80 — _multi_base_isolation flag set once at v3.15.56 init handshake; ETH/USDC bot init'd alone, sibling ETH/USDT added later, flag never re-evaluated; ETH/USDC kept reading shared exchange pool and froze on phantom deficit when ETH/USDT sold

R74 — Bypass Caller Audit (BCA)

tier: core · category: Safety · state: **active** · introduced: 3.15.81 · session: Session 26

Any code path documented as 'bypasses everything' (typically the manual-fire / operator-sovereign path) **MUST** be audited at every CALL SITE that is not directly operator-initiated. Automated paths that reuse the manual-fire trade machinery **INHERIT** the bypass posture by accident — they get the 'operator is sovereign' justification without the operator's consent. Each non-operator caller of a 'bypasses everything' function must explicitly re-apply the safety gates (hysteresis, profitability, dust-floor, etc.) that matter for its

specific context. Document each retained bypass and each re-applied gate in the call-site comment.

Canonical-failure MEMs: v3.15.79 — cartridge gate (automated) reused `_execute_manual_rebalance` and inherited 'bypasses hysteresis' from the operator-sovereign justification; produced fee-thrash round-trip that lost operator money

R75 — Pivot Reference Discipline (PRD)

tier: core · category: Reference Selection · state: **active** · introduced: 3.15.81 · session: Session 26

When a gate uses a price-distance check (interval, fee, hysteresis), the REFERENCE point for the distance must match the operator's mental model of the trade event. Two reference styles exist: (a) FIXED reference — the price at the last fill, persists across all subsequent ticks until a new fill; (b) PIVOT reference — the price at the moment a state condition was met (e.g., delta crossed zero into opposing territory), captured fresh each time the state arms. Choose the style that matches the operator's design intent. In oscillating markets, PIVOT references flicker but track the actual swing the trader cares about; FIXED references freeze and can make subsequent gates irrelevant or over-restrictive. Document the choice in the gate's docstring.

Canonical-failure MEMs: v3.15.77 — pre-fix opposing-direction hysteresis used `_last_trade_price` as fixed reference; operator wanted swing-pivot semantics; fix changed reference style to PIVOT

R76 — Defined-Means-Wired (DMW)

tier: core · category: Wiring · state: **active** · introduced: 3.15.81 · session: Session 26

Any class member, writer, subscriber, handler, or other side-effect-producing component that is constructed in `__init__` MUST have a verified caller. A writer with no writer-of, a subscriber with no event source, a handler with no dispatch site — these are bugs even if the code compiles and runs. The discipline: each side-effect component needs an explicit wiring test (static-text pin OR behavioral pin) that verifies the call path exists end-to-end. Constructing without wiring is a hidden orphan that the operator only discovers when the expected side-effect (logs, persisted state, GUI update) does not appear.

Canonical-failure MEMs: v3.15.78 — LogManager constructed an NDJSONWriter for `logs/real_market/trade.log` in `__init__`, but no caller invoked `log_trade()`; `trade.log` silently empty across every operator session until they noticed and reported

R77 — Surface Binding Resolution (SBR)

tier: core · category: Static Discipline · state: **active** · introduced: 3.15.97 · session: Session 26

Every Name reference inside a function or method body MUST resolve to one of: (a) an explicitly self-prefixed instance attribute, (b) a function parameter, (c) a name assigned earlier in the current function on every reachable code path, or (d) a name retrieved via `getattr(self, "name", default)` for optional state. Bare-name references that look like locals but rely on Python promoting an assignment-elsewhere-in-the-function to function-scope are forbidden. The `static_audits.audit_static_unbound_locals` AST scan enforces this at parse time; the `test_static_audits.test_no_static_unbound_locals_in_src` CI test pins the count at zero. Edits to any method longer than ~50 lines must run the audit before ship.

Depends on: R28

Canonical-failure MEMs: v3.15.66 — RAVE-bot bullseye_upper bare-name UnboundLocalError on first signal, v3.15.92 — Smart Cartridge engine introduced 9 bare bb_result references; UnboundLocalError on every cartridge calibration call, v3.15.94 — scrumming_bot.py:4437 SCRUM-blocked debug f-string with bare _phantom_locked; bots could not start (Uptime 0s), v3.15.97 — paper_trader_tab.py:_start() bare pair before assignment; grid_bot.py:_execute_initialisation() bare lv × 2 referenced before for-loop

7. Episodic Memory + EDIT_LOG + retag-by-rule

The memory layer is the part of SADP that makes continuity across session boundaries mechanically possible. It has two files plus a query convention. As of this writing, **sadp/EPISODIC_MEMORY.json** holds 453 entries totaling approximately 1.59 MB, and **sadp/EDIT_LOG.jsonl** holds 3,730 lines totaling approximately 0.95 MB. Both files are checked into version control, schema-validated by tests, and consulted by every cold-read protocol.

EPISODIC_MEMORY.json schema

Each entry in the Episodic Memory store is a JSON object with the following fields. The schema is enforced by R20 SVG (Schema-Validated Governance) regression tests; a malformed entry fails the release gate.

Field	Type	Purpose
id	string (MEM-N)	Monotonic identifier; N is unique
session	int	Session number when the entry was created
date	ISO date	When the entry was created
version	semver	Acervator version the entry was attached to
sadp_version	semver	SADP framework version at creation
category	string	Topic taxonomy slug for query/filter
keywords	list of strings	Searchable secondary tags
rules_implicated	list of rule IDs	Every rule the entry exercises or is governed by
patent_flag	bool	Whether the entry touches patentable subject
title	string	One-sentence summary
evidence_source	string	File paths, line numbers, audit doc references — the citations
summary	string (long)	Free-text body capturing what was learned, why it matters, what is queued

EDIT_LOG.jsonl format

Every cascade writes one or more rows to EDIT_LOG.jsonl. The log is append-only — R3 mandates that no row is ever modified or deleted in place. Each row is a JSON object on its own line with these fields:

- **timestamp** — ISO-8601 UTC timestamp
- **session** — session number
- **version** — Acervator version this row belongs to
- **file** — the file (or files) modified, comma-separated
- **line_range** — what part of the file changed, free-text
- **operation** — add / modify / remove / add+modify
- **authority** — MEM reference + operator-directive citation (R6 CEM compliance)
- **purpose** — short description of why the change was made

Retag-by-rule query pattern

The most common query against the memory layer is: 'show me every MEM that implicated rule X'. Because every MEM has a `rules_implicated` array, this query is a single linear scan of the JSON file. The same query against the EDIT_LOG is a scan for the rule ID inside the authority field. Together, these queries surface the operational history of any single rule — every time it was exercised, every time it was strengthened, every time a failure-MEM cited it as the canonical case. Sample retag-by-rule volumes as of v3.22.8:

Rule	MEMs implicating	Why so many
R28 DCA	60+	Every documentation cascade artifact
R57 EPM	30+	Production-engine ↔ simulation parity cascade-gate; every Extractor or RAIntSimBat ship triggers it
R68 First Principles	20+	Every bug audit that surfaced the speculative-fix pattern
R70 RCN	15+	Context-footer cadence references
R72 OTSSOT	10+	Hardcoded-override fixes since Session 53
R77 SBR	4+	UnboundLocalError ships in Session 68

Cold-read protocol consumption

R69 cold-read protocol bounds session-opening orientation to approximately 2-3% of the AI's context budget. Within that budget, the memory layer is the primary source. A typical cold-read reads: (1) the auto-memory MEMORY.md file (operator + project state pointers); (2) the docket (active queue); (3) the last ~5-10 MEMs in EPISODIC_MEMORY.json (recent context); (4) targeted retag-by-rule queries for the rules implicated in the upcoming task. The memory layer is engineered to make each of these queries fast.

8. Archetype Library + persistent leads + documentation-quality pair

Acervator's review process uses a library of professional-discipline archetypes as lenses for evaluating work. The library lives in `sadp/ARCHETYPE_LIBRARY.json` (42.7 KB) and is documented in `sadp/MULTI_AGENT_COORDINATION.md` (23.1 KB). Each archetype defines a structured reviewer role with a lens, domain tags, standards, canonical questions, and explicit failure modes. The library is the SADP-side equivalent of multiple-perspective design review (Brooks 1975), Six Thinking Hats (de Bono 1985), or red-team / blue-team drills.

Two tiers — core and domain

Archetypes fall into two tiers. **Core archetypes** apply broadly across software projects: correctness, architecture, performance, security, quality, documentation. They are candidates for transplant to any SADP-adopting project. **Domain archetypes** apply when the project's domain tags overlap with the archetype's tags. The Acervator instantiation ported L1-L10 (the trading/fintech/TA leads) to the domain tier in v3.21.0 with the slug-based architecture, so future SADP adoptions can leave the trading leads behind and add their own.

Recent archetype additions — v3.22.x

Six archetypes were added during the v3.22.x manual-sharpening arc to address gaps the operator surfaced. They are designed to critique not flatter — every entry's first `canonical_failure_mode` is the meta-failure-mode 'bland praise is the failure mode'.

Archetype	Code	Inspirations	Primary lens
Technical Writer Lead	C7	Zinsser, Tufte, Diataxis	Clarity, signposting, audience-named
Technical Analyst Lead	C8	Operator-inspired pair-partner	Source-of-truth verification, default introspection
Quantitative Analyst	Q1	Derman, López de Prado, Engle	Sharpe / Sortino / Calmar, buy-and-hold counterfactual
AI-Assisted Dev Practitioner	AI1	Simon Willison, Karpathy, Goodside	Pair-programming discipline, prompt economy
AI Safety / Alignment	AI2	Anthropic Constitutional AI, M.T. Ribeiro	Failure-mode taxonomy, adversarial framing
AI Research Engineer	AI3	Karpathy nanoGPT, P. Liang HELM, Olah	Empirical framing, what-is-not-yet-demonstrated

Archetype	Code	Inspirations	Primary lens
Innovation / Value Analyst	AI4	Helmer, Christensen, Diataxis	Prior-art scorecard, novelty-vs-restatement
Simulation Engineer	SIM 1	López de Prado, Carmack, Engle	Limits-first framing, what-the-sim-does-not-model

Persistent leads and Departmental Lead Rotation

Not every archetype is consulted on every cascade. R11 DLR (Departmental Lead Rotation) names the practice: each cycle, a subset of archetypes is appointed as the primary review panel; the rotation surfaces lenses that would otherwise be neglected. Persistent leads are the archetypes that anchor the panel across rotations — typically C1 Correctness + C2 Architecture for code-change cascades, C7+C8 for documentation cascades, Q1 for empirical-evidence ships. The pattern was documented in `sadp/MULTI_AGENT_COORDINATION.md §5` and is the subject of MEM-446 (archetype library structural test).

Documentation-quality pair (C7 + C8)

C7 Technical Writer + C8 Technical Analyst form a paired-partner relationship modeled on the operator's directive that documentation should be both readable (C7's lens — clarity, signposting, audience-named) and source-of-truth-accurate (C8's lens — every default verified, no stale tables, live introspection over hand-authored repetition). The pair anchors all manual-sharpening cascades in the v3.22.x arc. The cascade rituals for these two leads include: introspect the BotConfig dataclass for every default named in prose, cite the source file + line at the bottom of each section, and re-render the topic-matrix tool to verify cross-Part presence matches the intended audience tiering.

Structural test on the library

MEM-446 introduced `tests/test_archetype_library_v3_21_0.py` — a structural test verifying that every archetype entry has a non-empty slug, code, name, lens, and `canonical_failure_modes` array, and that no two archetypes share a slug or code. The test is the labels-vs-derived-truth pin for the archetype library: the labels (archetype metadata) must remain structurally well-formed even as the library grows.

9. Labels-vs-derived-truth structural test family

A specific class of bug surfaces repeatedly in SADP-governed work: a label or a state flag asserts something the underlying derivation does not actually support. The labeled state is what the code or the documentation CLAIMS to be true; the derived truth is what an independent re-computation says is ACTUALLY true. When they disagree, the labeled state is wrong — and because it is labeled, the wrongness is invisible until someone explicitly checks. The labels-vs-derived-truth structural test family is the specific machinery SADP uses to catch this class of bug.

Family members

MEM	Test file	Label	Derived truth
MEM-428	test_bot_config_wizard_parity_v3_20_82.py	Wizard 'page valid' flag	Re-validation against BotConfig at the moment the flag is set
MEM-429	test_session_integrity_v3_20_83.py	Session-file CLOSE marker	Cross-check that EDIT_LOG + CHANGELOG + MEM trio agrees
MEM-430	test_banner_triple_parity_v3_20_84.py	Banner version in src/__init__.py · main.py · docs/AUDITS.md	All three locations must agree
MEM-433	test_manual_topic_matrix_v3_20_87.py	Topic 'covered in expected_home Part' assertion	Re-scan of every Part PDF source for the topic's aliases
MEM-445	test_claims_audit_v3_22_4.py	Claims-audit puffery ceilings (HIGH / MEDIUM / LOW)	Re-scan of all Part builders for puffery-pattern matches
MEM-446	test_archetype_library_v3_21_0.py	Archetype library entry metadata	Schema-conformance check + uniqueness of slug + code
MEM-446 (b)	test_concept_novelty_v3_21_5.py	'Novel concept' assertion in cascade prose	Cross-check against prior MEM corpus for prior introduction

Ratchet pattern

Each member of the family is wired as a **ratchet** — the test has a numerical ceiling (count of allowed divergences) that monotonically decreases as the documentation matures. When a cascade closes drift, the ceiling drops in the same cascade. When a cascade introduces new drift, the ratchet FAILS — the release gate refuses to ship until the drift is either closed or the ceiling is explicitly raised with a written rationale. The rationale is the part that matters: an unexplained ceiling raise is a code smell visible in the diff. An explained one is a documented design tension.

Why this family matters more than typical regression tests

Conventional regression tests pin behavior — the code does X given input Y. Labels-vs-derived-truth tests pin **consistency** — the code's self-description (a label, a flag, a manifest, a banner) matches what the code actually does. The failure mode they catch is harder to surface than a behavior regression because it is invisible to a user until the labeled state matters. A wizard page marked 'valid' that actually contains an invalid combination of settings will not fail until the bot tries to run the combination. The structural test fails at parse time, not run time, which is the entire point.

10. Cascade-gate enforcement — PreToolUse hook + release-gate sovereignty

SADP rules are guardrails. Cascade gates are the machinery that REFUSES to let a ship close when a guardrail is violated. They are the enforcement layer on top of the rule layer. There are three primary cascade gates as of SADP v1.92.

Gate 1 — the release gate (tools/check_release_readiness.py)

This is the canonical gate. It runs the full test suite, validates the banner-triple agreement (R7 BTM), checks the test-count freshness pin (R8), and reports a single line: **[OK] Release-ready (vX.Y.Z, N tests)** or **[FAIL]** with an enumerated failure list. R2 PRM mandates that no banner-version bump is permitted without the [OK] reading. Operator directive 2026-05-31: 'FIX IT AND NEVER DO THIS AGAIN' — this rule was hardened after a ship in which banners were bumped while the suite had eighteen failures. Since the hardening, no banner-without-gate failure has recurred.

Gate 2 — Engine Parity Mandate (R57 EPM)

R57 EPM is a cascade-level gate: edits to the production Extractor engine without corresponding parity edits to the RAIntSimBat simulation engine are refused. The rule is enforced by a contract test that scans the diff and fails the suite if Extractor-side files changed without RAIntSimBat-side files changing in the same cascade. R57 was introduced after Session 56's discovery that production behavior had silently diverged from simulation behavior over multiple ships — the simulation was no longer telling the truth about what production would do. The gate is now the primary defense against that divergence class.

Gate 3 — PreToolUse hook layer (planned + partially implemented)

The third gate operates at a finer-grained timescale than release time. It is a PreToolUse hook in the AI pair programmer's tool-call layer: before a tool call executes (Bash, Edit, Write), the hook inspects the intended action against the active rule set and can refuse, warn, or permit. The intended use case is preventing actions that would obviously violate a rule — e.g., a banner-bump Edit without a release-gate-pass record in the same session, or a Write to a file the test pin says is frozen. The hook layer is partially implemented as of v3.22.8; the release-gate refusal is fully wired, the rule-set inspection is in design.

Doc-audit ratchet — the supplementary gate

Beyond the three primary gates, the doc-audit ratchet (`sadp/_tools/doc_audit.py` + `tests/test_doc_audit_baseline.py`) is a documentation-drift cascade gate. `PART_POLICY` in the tool maps each manual Part to a lag-floor ceiling (how many version increments the Part may trail the current banner). Floors ratchet down as Parts are refreshed. The release gate calls into the doc-audit ratchet; an over-ceiling floor refuses the ship. As of v3.22.7, Part 10 was added with `lag_floor 0 + min_mentions 5 + content_heavy=True`.

11. Token Timer per-action accounting + grounding layers

Token Timers are the youngest grounding layer in SADP. They address a failure mode the older layers do not catch: drift over effort. Rules and freshness checks fire when a violation happens or when a file becomes stale relative to another file. Token Timers fire when too much work has accumulated since the last grounding event — when the AI pair programmer has been shipping for a while without consulting the rules and speculative cycles are starting to compound. The mechanism is implemented in **sadp/token_timer.py** (646 lines) and the per-action log lives in **sadp/TOKEN_TIMER_LOG.jsonl**.

Three SADP grounding layers

Layer	Trigger	Example
Rules	Action-event triggers the rule	R2 PRM fires when a banner-bump Edit is staged without a release-gate pass in the same cascade
Freshness	File X becomes older than file Y	AGENTS.md staleness vs sadp/ source files; doc-audit ratchet floor lag against current banner
Token Timers	N actions without a grounding event	After K cold_read / hypothesis / fix_shipped events, the next operator_correction or shipped fix-without-grounding logs a TIMER_FIRED row

Per-action API

The `token_timer` module exposes a `record()` call that the AI pair programmer is expected to invoke on every grounding-relevant action: `cold_read` at session start, `hypothesis` when proposing a fix without yet having read the code, `fix_shipped` after a cascade closes, and `operator_correction` when the operator surfaces a misframing or factual error. Each call appends one row to `TOKEN_TIMER_LOG.jsonl` with a timestamp + action type + optional context payload. The module also exposes a `--check` CLI that reads the log and reports whether any timer threshold has been exceeded.

Honest acknowledgment — what Token Timers do NOT measure

Token Timers are misleadingly named. They do not measure tokens. The AI pair programmer cannot reliably introspect its own token budget in real time. What the Timers measure is **action count between grounding events** — a proxy for token cost that is observable and recordable. The operator's original framing ('Token Timers') treats the action count as a stand-in for the drift-over-effort signal the layer is actually trying to surface. The name is preserved in honor of the genesis; the mechanism is engineered to work without true token introspection.

R70 RCN — the in-session counterpart

R70 Recurring Context Notification is the per-turn footer that reports the AI's context budget as a percentage. R70 fires on every response; Token Timers fire on grounding events. The two layers are complementary: R70 surfaces context-exhaustion as it approaches; Token Timers surface drift-over-effort across the session. Both layers are honest about uncertainty — R70's percentage is an operator-facing estimate, not a precise measurement; the operator can correct the estimate at any time (and has, during this very ship's development).

12. Multi-Agent Coordination Spec v0.1

`sadp/MULTI_AGENT_COORDINATION.md` (23.1 KB) is the v0.1 specification for how SADP handles work performed by multiple AI agents — not yet in production at version 1.92 but designed and partially implemented. The spec was extracted from a series of MEMs (MEM-446 archetype library + persistent leads + MEM-453 cross-archetype review pattern + v3.22.x documentation-quality pair) into a single document that can be consulted standalone.

Sections of the spec

- **§1 Persistent Leads.** The Departmental Lead Rotation (R11 DLR) implementation — which archetypes anchor each rotation, how rotations are decided, when a lead is promoted from rotation to persistent.
- **§2 Adaptive Lead Spawning.** The query against `ARCHETYPE_LIBRARY.json` that selects which archetypes apply to a given work item based on project domain tags + affected subsystem tags.
- **§3 Multi-Agent Cascade Atomicity.** How R5 (cascade atomicity) generalizes when more than one AI agent is participating — the entire cascade still ships or fails as a unit, but the unit-of-cascade may include work products from multiple agents.
- **§4 Conflict Resolution.** When two agents disagree, the operator is the tie-breaker; SADP does not auto-arbitrate. The spec documents the disagreement-surfacing convention so the operator can decide quickly.
- **§5 Documentation-Quality Pair.** The C7+C8 documentation-quality pair (added in v3.22.x) as a persistent-lead instance — every documentation cascade passes through both lenses before shipping.
- **§6 Prior-Art Acknowledgment.** A table mapping every Multi-Agent Spec primitive to its prior art (Brooks 1975, de Bono 1985, modern actor-system patterns). Added in v3.22.x parallel to the SADP-level prior-art table.

Not yet in production

The spec is at version 0.1 because SADP has not yet been exercised against true multi-agent work. The Acervator instantiation has one operator + one AI pair programmer; the archetype library is consulted as lenses by that single AI, not as separate agents. Whether the cascade-atomicity generalization actually works under multiple concurrent agents is an open question. The spec is published in advance of empirical validation so that adopters can plan, not so that adopters can rely on it.

13. Developmental story — sessions, ships, and rule births

SADP grew across approximately seventy sessions of Acervator development over fifteen months. The story below collapses that arc into four eras. Each era is named for the dominant concern it surfaced and lists the rules it birthed and the MEMs that anchored the era's lessons.

Era 1 — Founding (Sessions 22-25)

Trigger: a fix-and-forget loop on a regression that had been claimed-fixed twice before. Dominant concern: continuity across sessions. Rules born: R1 lock-state default + R3 append-only EDIT_LOG + R4 MEM-N notation + R5 cascade atomicity + R6 CEM + R7 BTM banner triple. By the end of Session 25, the rule count was 65 and the protocol had a name: SADP, version 1.0. Anchor MEMs: MEM-001 through MEM-066 (the founding corpus). Anti-pattern audit took place at Sessions 23-25; resulting rules absorbed the patterns into preventatives.

Era 2 — Maturation (Sessions 26-37)

Dominant concern: session-anti-pattern lesson capture. Sessions kept surfacing the same kinds of mistakes — claims-without-citation, opportunistic confidence, premature closure. R66 anti-pattern audit discipline and R67 session-anti-pattern lesson capture were born here. The doc-audit instrument (R25 DARP) was built in this era to give the protocol a measurable drift signal. SADP versioned to 1.20. Anchor MEMs: MEM-067 through MEM-200.

Era 3 — First Principles & Observability (Sessions 38-52)

Dominant concern: bugs that resisted speculative fixes. R68 First Principles Come First was born from a session in which three speculative fixes had been shipped for a bug whose root cause had not been audited. The cold-read protocol (R69) and the Recurring Context Notification (R70) followed, extending earlier R35 ritual-point reporting to per-turn context-budget visibility. The L8 OOS attribution battery (v3.19.21) was the empirical instrument that surfaced drift in the trading engine's out-of-sample behavior. Anchor MEMs: MEM-201 through MEM-340.

Era 4 — Discipline Maturation (Sessions 53-70)

Dominant concern: specific failure modes as the system grew more complex. R71 Observability + R72 OTSSOT + R73 State Management + R74 Safety + R75 PRD + R76 Wiring + R77 SBR — seven rules covering ad-hoc state mutation, hardcoded overrides of operator-tunables, gate-stack bypass, reference-point ambiguity, runtime activation gaps, and static-binding failures. R57 EPM (Engine Parity Mandate) was hardened in this era after the production / simulation divergence became visible. The v3.22.x manual-sharpening arc — the arc this very document is part of — happened in this era. SADP versioned to 1.92. Anchor MEMs: MEM-341 through MEM-458.

Rule-birth cadence

Across the four eras, the rule birth rate averages approximately 1.1 rules per session. The rate is not uniform — Era 1's anti-pattern audit produced a burst (roughly 22 rules per 3 sessions), Era 2's maturation produced a slower steady cadence, Era 3 produced a small burst around the First Principles realization, Era 4 has produced a roughly linear cadence with occasional clusters around specific failure modes. Whether the cadence is sustainable past Era 4 is one of the open empirical questions named in section 3.

14. Glossary (standalone-complete)

Every term used in this document is defined below. The glossary is designed to be standalone-complete — a reader who knows software engineering but has not encountered SADP before should be able to look up any unfamiliar term here and find an explanation that does not assume Acervator context.

Term	Definition
Archetype	A structured professional-discipline reviewer role — a lens applied to work for evaluation. Defined in <code>sadp/ARCHETYPE_LIBRARY.json</code> with metadata: slug, code, name, domain tags, standards, canonical questions, canonical failure modes. Used by the persistent-leads and Departmental Lead Rotation patterns.
Banner Triple	The three locations where Acervator's version number must agree: <code>src/__init__.py</code> , <code>__version__</code> , <code>main.py</code> , <code>current_version</code> , <code>docs/AUDITS.md</code> 'Current version'. R7 BTM mandates consistency; MEM-430 structural test pins it.
Cascade	A unit of shipped work. The cascade includes the code change, CHANGELOG entry, MEM entry, EDIT_LOG row, banner bump, release-gate pass, and zip artifact. R5 mandates atomicity — all ship or none ship.
Cold Read	The session-opening protocol that orients the AI pair programmer to current state. Bounded by R69 to approximately 2-3% of context budget. Reads auto-memory, docket, recent MEMs, and targeted retag-by-rule queries.
Departmental Lead Rotation (DLR)	R11 — the practice of rotating which archetypes anchor the review panel cycle by cycle, so lenses that would otherwise be neglected get applied.
Docket	<code>sadp/DOCKET.md</code> — the cross-session deferral queue. Holds work items that surfaced during a session but were not the session's focus.
Doc-Audit Ratchet	<code>sadp/_tools/doc_audit.py</code> + <code>PART_POLICY</code> — a documentation-drift cascade gate. Each manual Part has a lag-floor ceiling that ratchets down as Parts are refreshed. R25 DARP names the entry/exit protocol.
EDIT_LOG	<code>sadp/EDIT_LOG.jsonl</code> — the append-only event log. Every cascade writes one or more rows. Schema: timestamp, session, version, file, line_range, operation, authority, purpose. R3 mandates append-only.
Engine Parity Mandate (EPM)	R57 — the cascade-gate that refuses Extractor edits without corresponding RAIntSimBat parity edits. Prevents production / simulation divergence.
Episodic Memory	<code>sadp/EPISODIC_MEMORY.json</code> — the schema-validated MEM store. As of v3.22.8, 453 entries. Each entry is rule-tagged for retag-by-rule queries.
First Principles Come First (FPCF)	R68 — when a bug resists multiple speculative fixes, STOP shipping fixes. Audit the actual code state. Add observability. Let the system tell you what is true.
Grounding Event	An action that re-anchors the AI pair programmer to reality: <code>cold_read</code> , <code>hypothesis</code> (with citation), <code>fix_shipped</code> , <code>operator_correction</code> . Recorded by Token Timers as a per-action log row.

Term	Definition
Hop	A session-spanning continuity protocol. The HOP file (currently HOP5) captures state across sessions. R12 HC mandates that the hop file accompanies every session open/close.
Labels-vs-Derived-Truth	A class of bug: a label or flag asserts something the code does not actually do. The structural test family (MEM-428 to MEM-446) catches the divergence.
LOCKED	Default rule state. The rule is in force and must be honored. Contrast UNLOCKED (deliberately relaxed) and EXPIRED (deprecated).
MEM-N	Episodic Memory entry identifier. N is a monotonic integer. The notation is established by R4.
N=1	The framing SADP uses for its single-project status. Acervator is the N=1 reference implementation. Transferability to a second N is an open empirical question (section 3).
Operator-Tunable Sole Source of Truth (OTSSOT)	R72 — when a feature is exposed to the operator as a tunable, the operator's value MUST be the sole source of truth. No hardcoded constant may override it.
Persistent Lead	An archetype that anchors the review panel across rotations. Documented in MULTI_AGENT_COORDINATION.md §1.
Pre-Release Mandate (PRM)	R2 — no banner-version bump is permitted without [OK] Release-ready from tools/check_release_readiness.py.
Recurring Context Notification (RCN)	R70 — per-turn context-budget reporting by the AI pair programmer. Footer format: context percentage + kilotokens / total.
Release Gate	tools/check_release_readiness.py — the sovereign cascade gate. Runs the full suite, validates banner triple, reports [OK] Release-ready or [FAIL].
Retag-by-Rule	Query pattern against the memory layer: 'show me every MEM that implicated rule X'. Linear scan; O(n) over the MEM corpus.
SADP	Structured AI Development Protocol. The framework this document describes.
Structural Test	A test that pins consistency rather than behavior. Verifies that a label, flag, or manifest matches the derived truth of what the code actually does. The labels-vs-derived-truth family is the canonical example.
Token Timer	sadp/token_timer.py — the grounding-event accounting layer. Counts actions between grounding events as a proxy for drift-over-effort. Misleadingly named (does not measure tokens directly).
Tier	Rule classification: core (transplant candidate) vs project_overlay (Acervator-specific). See section 5.

End of Part 10. SADP v1.92 — a single-project artifact at version 1.x. Seventy-seven rules, 453 MEMs, an append-only EDIT_LOG, an archetype library, a structural test family, three grounding layers, and three cascade gates. Built incrementally over approximately seventy

sessions on the Acervator project. Whether SADP is a framework — meaning, whether it survives transplant to a different project, language, model, or practitioner — is an empirical question that has not yet been answered. The current evidence supports the claim that SADP works for Acervator. The current evidence does NOT support the stronger claim that SADP works in general. This document is structured around that distinction.